Ponto De Conhecimento Tecnológico

SÉRIE 2 — SQL SERVER

HANDS-ON #08
Transações, Concorrência e Deadlocks

Guia técnico laboratorial | SQL Server 2019 Express | SIManagerDB

Versão de revisão — 05/11/2026 | Conteúdo sujeito a validação final

Base documental: execução e resultados partilhados ao longo dos oito exercícios. Não são atribuídas medições que não tenham sido recolhidas.

Índice

1. Introdução, objetivos e enquadramento

2. Ambiente técnico e SIManagerDB

3. Fundamentos de transações e propriedades ACID

4. Gestão de transações e tratamento de erros

5. Níveis de isolamento e anomalias de leitura

6. Bloqueios e diagnóstico com DMVs

7. Deadlocks: reprodução e resolução

8. Prevenção e comparação de cenários

9. Conclusões, boas práticas e anexos técnicos

1. Introdução, objetivos e enquadramento

O acesso concorrente a dados é uma característica essencial dos sistemas empresariais. O SQL Server coordena operações simultâneas através de transações, níveis de isolamento e bloqueios, permitindo preservar a consistência sem eliminar a concorrência.

Este HANDS-ON documenta oito exercícios executados na base SIManagerDB, com duas sessões operacionais (SPID 57 e 59) e uma sessão de diagnóstico (SPID 52). Foram observados bloqueios de leitura, leituras sujas, leituras não repetíveis, um deadlock real e uma estratégia de prevenção baseada na ordem de acesso.

Objetivos de aprendizagem

• Executar BEGIN TRAN, COMMIT e ROLLBACK, verificando o efeito das alterações.

• Tratar falhas transacionais com TRY...CATCH, XACT_STATE() e XACT_ABORT.

• Comparar READ COMMITTED e READ UNCOMMITTED em sessões concorrentes.

• Reproduzir Dirty Read e Non-Repeatable Read.

• Diagnosticar sessões bloqueadas através de sys.dm_exec_requests.

• Reproduzir o erro 1205 e observar o ROLLBACK automático da vítima.

• Demonstrar uma ordem consistente de aquisição de bloqueios.

Metodologia e limites

Os testes utilizaram tabelas laboratoriais separadas dos dados funcionais. Os valores registados representam execuções observadas, não benchmarks de produção. A ausência de deadlock num teste não garante a sua impossibilidade noutros planos de execução ou cargas concorrentes.

2. Ambiente técnico e SIManagerDB

Componente	Configuração validada
Motor	SQL Server 2019 Express, 15.0.2190.7 (64-bit)
Base de dados	SIManagerDB, ONLINE; compatibilidade 150; recuperação SIMPLE
READ_COMMITTED_SNAPSHOT	OFF (0)
ALLOW_SNAPSHOT_ISOLATION	OFF
Sessão A	SPID 57
Sessão B	SPID 59
Sessão C	SPID 52 (diagnóstico)

A SIManagerDB contém 25 aplicações, 50 000 incidentes, 12 000 pedidos de serviço e 25 000 intervenções, de acordo com a verificação inicial. Para os testes concorrentes foram usadas dbo.H08_TesteIsolamento e dbo.H08_TesteDeadlock.

Estrutura das tabelas laboratoriais

```
CREATE TABLE dbo.H08_TesteIsolamento (
    Id INT NOT NULL PRIMARY KEY,
    Descricao VARCHAR(100) NOT NULL,
    Valor INT NOT NULL
);
-- Registo inicial: (1, 'Valor inicial', 100)

CREATE TABLE dbo.H08_TesteDeadlock (
    Id INT NOT NULL PRIMARY KEY,
    Descricao VARCHAR(100) NOT NULL,
    Valor INT NOT NULL
);
-- Registos iniciais: (1, 'Registo A', 100),
--                    (2, 'Registo B', 200)
```

Os scripts apresentados são excertos representativos dos procedimentos validados. A sequência de execução entre sessões é determinante para reproduzir os fenómenos.

3. Fundamentos de transações e propriedades ACID

Uma transação agrupa operações que devem ser tratadas como uma unidade lógica. BEGIN TRAN inicia uma transação explícita; COMMIT confirma as alterações; ROLLBACK anula as alterações ainda não confirmadas.

Propriedade	Significado operacional
Atomicidade	Todas as operações da transação são confirmadas ou anuladas como unidade.
Consistência	As restrições e invariantes da base de dados devem ser preservadas.
Isolamento	O resultado das operações concorrentes depende do nível de isolamento e dos mecanismos de controlo de concorrência.
Durabilidade	Após confirmação, os efeitos da transação persistem segundo as garantias do motor.

Exercício 2 — COMMIT e ROLLBACK

Com uma tabela temporária local, a descrição inicial foi alterada dentro de uma transação. Antes do ROLLBACK observou-se “Alteracao temporaria”, com @@TRANCOUNT = 1; após o ROLLBACK voltou a “Estado inicial”, com @@TRANCOUNT = 0. Num segundo teste, COMMIT preservou “Alteracao confirmada”.

```
BEGIN TRAN;
UPDATE #TesteTransacao
SET Descricao = 'Alteracao temporaria'
WHERE Id = 1;
SELECT @@TRANCOUNT AS TransacoesAbertas;
ROLLBACK TRAN;
-- A linha regressa ao estado inicial.
```

4. Gestão de transações e tratamento de erros

Exercício 3 — TRY...CATCH e XACT_STATE()

O exercício provocou deliberadamente uma violação de chave primária ao inserir um identificador duplicado. O SQL Server devolveu o erro 2627. No bloco CATCH foram observados XACT_STATE() = -1 e @@TRANCOUNT = 1, sinalizando uma transação não confirmável. O ROLLBACK repôs o estado anterior, com XACT_STATE() = 0 e @@TRANCOUNT = 0.

```
SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRAN;
    -- Executar operação válida e depois INSERT
    -- com chave primária duplicada.
    COMMIT TRAN;
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS NumeroErro,
           ERROR_MESSAGE() AS Mensagem,
           XACT_STATE() AS EstadoTransacao,
           @@TRANCOUNT AS TransacoesAbertas;
    IF XACT_STATE() <> 0 ROLLBACK TRAN;
END CATCH;
```

Nota: este excerto ilustra o padrão de tratamento; o INSERT específico do laboratório não é reproduzido integralmente por não constar dos resultados transcritos nesta fase. XACT_STATE() = -1 impede COMMIT, sendo necessário ROLLBACK.

Cuidados de implementação

• Não confundir @@TRANCOUNT (profundidade de transações explícitas/implícitas) com XACT_STATE() (estado transacional).

• Limitar a duração das transações, sobretudo quando existem operações de rede ou interação humana.

• Garantir tratamento de exceções e limpeza de transações em todas as vias de erro.

5. Níveis de isolamento e anomalias de leitura

Exercício 4 — READ COMMITTED e Dirty Read

A Sessão A iniciou uma transação, alterou o valor de 100 para 500 e manteve-a aberta. A Sessão B, em READ COMMITTED, tentou ler o registo e obteve Msg 1222 após SET LOCK_TIMEOUT 5000. Em READ UNCOMMITTED, a mesma sessão leu 500, apesar da ausência de COMMIT. O ROLLBACK na Sessão A repôs 100, confirmado posteriormente pela Sessão B.

Momento	Sessão	Observação
Antes da alteração	A	Valor 100
UPDATE sem COMMIT	A (57)	Valor 500; transação aberta
READ COMMITTED	B (59)	Msg 1222 — lock request time out
READ UNCOMMITTED	B (59)	Valor 500 (Dirty Read)
ROLLBACK	A (57)	Valor reposto para 100
Leitura final	B (59)	Valor 100

```
-- Sessão B, enquanto a Sessão A mantém UPDATE sem COMMIT
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT Id, Descricao, Valor
FROM dbo.H08_TesteIsolamento WHERE Id = 1;
-- Resultado observado: Valor = 500
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Exercício 5 — Non-Repeatable Read

A Sessão A iniciou uma transação em READ COMMITTED e leu 100. Sem terminar essa transação, a Sessão B alterou o registo para 200 e executou COMMIT. Uma segunda leitura na Sessão A devolveu 200, apesar de decorrer na mesma transação. A Sessão A terminou com COMMIT e @@TRANCOUNT = 0.

Leitura	Valor observado	Condição
Primeira leitura na A	100	Antes do COMMIT da B
Atualização na B	200	COMMIT concluído
Segunda leitura na A	200	Mesma transação da primeira leitura

A experiência ilustra que READ COMMITTED impede leituras sujas, mas não garante que uma linha lida duas vezes conserve o mesmo valor. Em ambientes com READ_COMMITTED_SNAPSHOT ON, o mecanismo de leitura seria diferente; esta experiência foi realizada com a opção OFF.

6. Bloqueios e diagnóstico com DMVs

Exercício 6 — Bloqueio controlado

A Sessão A atualizou o registo para 300 sem COMMIT. A Sessão B tentou executar SELECT em READ COMMITTED e ficou suspensa. A Sessão C consultou sys.dm_exec_requests, identificando a Sessão 59 bloqueada pela Sessão 57.

Campo DMV	Valor observado
session_id	59
blocking_session_id	57
status	suspended
command	SELECT
wait_type	LCK_M_S
wait_time	59 821 ms
wait_resource	KEY: 14:72057594045267968 (8194443284a0)
database	SIManagerDB

```
SELECT r.session_id AS SessaoBloqueada,
       r.blocking_session_id AS SessaoBloqueadora,
       r.status, r.command, r.wait_type,
       r.wait_time, r.wait_resource
FROM sys.dm_exec_requests AS r
WHERE r.blocking_session_id > 0;
```

O tipo LCK_M_S corresponde à espera por um bloqueio partilhado. A identificação KEY indica um recurso de chave de índice. O tempo observado é uma fotografia do momento da consulta, não a duração total do bloqueio.

Após ROLLBACK na Sessão A, o valor regressou a 200. A consulta original da Sessão B terminou automaticamente e devolveu 200. Esta sequência confirmou a relação entre bloqueio, espera e libertação.

7. Deadlocks: reprodução, deteção e resolução

Exercício 7 — Dependência circular

A Sessão A iniciou uma transação e atualizou Id = 1 para 150. A Sessão B iniciou outra transação e atualizou Id = 2 para 250. Depois, a Sessão A tentou atualizar Id = 2 para 275 e ficou à espera; a Sessão B tentou atualizar Id = 1 para 175. Formou-se uma dependência circular.

Fase	Sessão A (57)	Sessão B (59)
1	X Lock no Id = 1	X Lock no Id = 2
2	Espera por Id = 2	Espera por Id = 1
Resolução	Sobrevive	Vítima: Msg 1205
Após deteção	1 transação aberta	0 transações abertas

Evidência do erro 1205

```
Msg 1205, Level 13, State 51
Transaction (Process ID 59) was deadlocked on lock
resources with another process and has been chosen
as the deadlock victim. Rerun the transaction.
```

O SQL Server anulou automaticamente a transação da Sessão B. A consulta de diagnóstico na Sessão C devolveu sessão 57 sleeping com uma transação aberta e sessão 59 sleeping com zero transações abertas. Após COMMIT na Sessão A, os valores finais foram Id = 1: 150 e Id = 2: 275.

A escolha da vítima não deve ser interpretada como uma regra fixa de seleção da sessão 59. Noutros cenários, o motor pode escolher outra transação, em função do custo de rollback e da prioridade de deadlock.

8. Prevenção e comparação de cenários

Exercício 8 — Ordem consistente de acesso

Os dados foram repostos para 100 e 200. A Sessão A atualizou primeiro Id = 1 para 150 e depois Id = 2 para 250, mantendo a transação aberta. A Sessão B seguiu a mesma ordem, tentando atualizar Id = 1 para 175 e Id = 2 para 275. A Sessão B ficou bloqueada no primeiro registo, sem dependência circular.

Após COMMIT na Sessão A, a Sessão B concluiu as duas atualizações. A validação registou @@TRANCOUNT = 1, Id = 1 com 175 e Id = 2 com 275. O COMMIT final da Sessão B terminou com @@TRANCOUNT = 0 e preservou os valores 175 e 275.

Critério	Exercício 7	Exercício 8
Ordem de acesso	A: 1→2; B: 2→1	A: 1→2; B: 1→2
Dependência circular	Sim	Não, no cenário observado
Efeito	Deadlock; Msg 1205	Blocking temporário
Resultado	B anulada; A confirmada	A e B confirmadas
Valores finais	150 / 275	175 / 275

Medidas adicionais de mitigação

• Aceder aos recursos numa ordem consistente em todos os caminhos de execução.

• Manter as transações curtas e evitar operações demoradas durante bloqueios exclusivos.

• Criar índices adequados para reduzir a abrangência e a duração de leituras e atualizações.

• Implementar repetição controlada de operações idempotentes após o erro 1205, com limites e backoff.

• Monitorizar deadlocks através de Extended Events, incluindo system_health, quando disponível.

• Avaliar READ_COMMITTED_SNAPSHOT de acordo com a semântica exigida e o impacto em tempdb; não o tratar como solução universal.

Representação comparativa dos valores finais

Cenário	Id = 1	Id = 2
Estado inicial	100	200
Após deadlock (COMMIT de A)	150	275
Após prevenção (COMMIT de B)	175	275

Esta tabela compara valores funcionais observados, não métricas de desempenho. Não foi medida a duração total do deadlock nem realizada uma comparação estatística de tempos de execução.

9. Conclusões, boas práticas e anexos técnicos

Os oito exercícios demonstraram o efeito direto da gestão transacional sobre a consistência e a concorrência. READ COMMITTED evitou uma leitura suja, mas permitiu uma leitura não repetível. READ UNCOMMITTED tornou visível um valor que viria a ser anulado. As DMVs permitiram identificar o bloqueio com precisão e o SQL Server resolveu um deadlock ao anular uma transação.

O último exercício mostrou que a ordem consistente de aquisição de bloqueios pode substituir uma dependência circular por uma espera temporária, permitindo que ambas as transações sejam confirmadas. Esta estratégia reduz riscos num padrão concreto, sem eliminar todas as causas de deadlock.

Matriz final de evidências

Exercício	Evidência confirmada
1	Ambiente SQL Server 2019 Express; SIManagerDB ONLINE
2	ROLLBACK repôs estado; COMMIT preservou alteração
3	Erro 2627; XACT_STATE() = -1; ROLLBACK
4	Erro 1222; Dirty Read 500; ROLLBACK para 100
5	Leitura 100 → 200 na mesma transação
6	SPID 59 bloqueada por 57; LCK_M_S; 59 821 ms
7	Erro 1205; vítima SPID 59; valores finais 150/275
8	Mesma ordem de acesso; valores finais 175/275; sem 1205

Anexo A — Consulta de verificação final

```
USE SIManagerDB;
GO
SELECT Id, Descricao, Valor
FROM dbo.H08_TesteDeadlock
ORDER BY Id;

SELECT Id, Descricao, Valor
FROM dbo.H08_TesteIsolamento
WHERE Id = 1;

SELECT @@SPID AS IdSessao,
       @@TRANCOUNT AS TransacoesAbertas;
```

Anexo B — Notas de segurança laboratorial

As tabelas H08_TesteIsolamento e H08_TesteDeadlock são específicas do laboratório. Não executar scripts de bloqueio ou deadlock em produção. A eliminação das tabelas laboratoriais deve ser feita apenas depois de guardar os resultados e confirmar que não existem sessões a utilizá-las.

Anexo C — Estado editorial

Documento preparado para revisão integral. Os resultados numéricos e os erros reproduzidos derivam das evidências fornecidas durante a execução. Os excertos de SQL são ilustrativos ou reconstituídos a partir dos comandos do laboratório; não constituem um arquivo completo dos ficheiros .sql originais. As capturas de ecrã originais não foram incorporadas nesta versão por não estarem todas disponíveis como ficheiros verificáveis no contexto atual.

Fim do documento — versão de revisão.